C PROGRAMMING — TEACHER'S NOTES

Pages 174–191 | Simple English explanations for teaching secondary-school students

Important: These notes follow the uploaded textbook pages in order. The aim is not to replace the textbook, but to turn each topic into a simple explanation you can use in class.

Coverage: Exercise 4.2; variables; declaration; global variables; function parameters; local variables; initialisation; const variables; updating variables; basic data types; constants and #define; type conversion and type casting; operators and expressions; arithmetic, relational, logical and assignment operators; operator precedence; increment/decrement; Exercise 4.6; and the introduction to Control Flow in C.

PAGE 174 — Exercise 4.2 and Variables

> **Teacher goal:** Start by checking the previous lesson, then move naturally into variables and how data is stored.

Exercise 4.2 — Revision

The page begins with questions about the basic structure of a C programme, syntax errors, printing Hello World, the purpose of `#include <stdio.h>`, and what happens when a program finishes executing.

Useful answers for teaching:

1. **main()** contains the main part where program execution begins.

2. The `printf()` statement needs a semicolon at the end. For example: `printf("The value of a is: %d\n", a);`

3. A simple Hello World program is:

```
#include <stdio.h>

int main()
{
printf("Hello, World!\n");
return 0;
}
```

4. `#include <stdio.h>` allows the program to use standard input/output functions such as `printf()` and `scanf()`.

5. When a C program finishes successfully, control returns from `main()`; `return 0;` commonly indicates successful completion.

What is a Variable?

Simple definition: A variable is a named place in computer memory used to store a value.

> **Easy classroom explanation:** “Think of a variable as a labelled box. The label is the variable name, and the box holds a value.”

```
int age = 20;
```

Here **int** tells C the type of data, **age** is the variable name, and **20** is the stored value.

The textbook also highlights four useful properties of variables: **name**, **size**, **lifetime**, and **scope**.

PAGE 175 — Declaring Variables

A variable must normally be declared before it is used. Declaration means creating the variable and specifying its data type.

```
int age;
```

Read this as: “Create a variable called age that stores an integer.”

General form: `data_type variable_name;`

```
int age;
float weight;
char grade;
```

A. Global Variables

A global variable is declared outside functions. It has global scope and can be accessed from parts of the program where it is visible.

```
#include <stdio.h>

int number = 10; // global variable

void printNumber()
{
printf("Number: %d\n", number);
}

int main()
{
printNumber();
return 0;
}
```

Simple example: Think of a notice on a school noticeboard. It can be seen by many people. A global variable is similarly available more widely than a local variable.

Important: The textbook warns that excessive use of global variables can make programs harder to understand and debug. Therefore, use them carefully.

B. Function Parameters

Parameters are variables listed in a function's header. They provide input to the function.

```
int add(int a, int b)
{
return a + b;
}
```

When we call `add(5, 3)`, 5 is passed to `a` and 3 is passed to `b`. The function adds them and returns 8.

Simple classroom sentence: “Parameters are the inputs that a function receives so that it can do its job.”

PAGE 176 — Function Parameters and Local Variables

Continue the previous explanation by showing that parameters work inside the function and are temporary for that function call.

```
#include <stdio.h>

int add(int a, int b)
{
return a + b;
}

int main()
{
int answer = add(5, 3);
printf("%d\n", answer);
return 0;
}
```

> **Step by step:** 1) main calls add(5,3). 2) 5 goes to a. 3) 3 goes to b. 4) a+b gives 8. 5) 8 is returned to answer. 6) printf displays 8.

C. Local Variables

A local variable is declared inside a function or block and can only be used within that scope.

```
#include <stdio.h>

void display()
{
int x = 10;
printf("Value of x: %d\n", x);
}

int main()
{
display();
return 0;
}
```

Here, `x` belongs to `display()`. Trying to use `x` directly in `main()` would cause an error because `x` is local to display().

> **Easy analogy:** A book kept inside one classroom is available to that classroom. It is not automatically available in another classroom. In the same way, a local variable is available only in its local scope.

PAGE 177 — Initialising a Variable

Initialisation means giving a variable its first value when it is declared.

```
int sum = 0;
int n = 100;
double pi = 3.14;
```

> Tell students: “Declaration creates the box. Initialisation puts the first value into the box.”

Why initialise? A variable that has not been given a suitable value may contain an unwanted value. Initialising variables makes the program predictable and easier to understand.

Local, Global and Static Initialisation

For a **local variable**, the initial value can be produced using an expression, including a calculation or function call.

```
void test()
{
int x = 5 + 3;
}
```

Global and static variables have stricter rules: their initial values must be constant expressions/values allowed for static storage. The important beginner idea is that global/static initialisation cannot simply depend on a value calculated at run time.

> **Teacher tip:** Do not overcomplicate this point for beginners. Make sure students first understand the difference between `int x;` and `int x = 10;`.

PAGE 178 — const Variables and Updating Variables

A `const` variable is intended to keep the same value after it has been initialised.

```
const int x = 10;
```

After this, trying to assign a new value such as `x = 20;` is not allowed.

> **Easy explanation:** “A normal variable is like a whiteboard: you can change it. A const variable is like a locked value: you set it once and should not change it.”

Why use const? It helps prevent accidental changes and is useful for values that should remain fixed, such as a maximum score or a mathematical constant.

Updating a Variable

Updating means changing the value of an existing variable.

```
int age = 15;
age = 20;
```

Step 1: age is created with 15. Step 2: age is changed to 20. This is allowed because age is a normal variable, not a const variable.

> **Remember:** `const int age = 15;` cannot later be changed with `age = 20;`.

Basic Data Types

C provides different data types for different kinds of values. The main types introduced here are `int`, `float`, `double`, and `char`.

PAGE 179 — Data Types, Constants and Type Conversion

Data type	Purpose	Simple example
int	Whole numbers	int age = 16;
float	Decimal numbers with single precision	float height = 5.9;
double	Decimal numbers with higher precision	double pi = 3.14159;
char	A single character	char grade = 'A';

Constants and #define Macros

A constant is a value that should not change during program execution. The page introduces two main ways to represent constants: `const` and `#define`.

```
const double PI = 3.1416;

#define PI 3.1416
```

Simple distinction: `const` creates a typed object whose value should not be modified. `#define` is handled by the preprocessor before compilation and replaces the macro name with its defined text.

Type Conversion and Type Casting

Sometimes one data type needs to be converted into another.

Implicit type conversion: the compiler performs the conversion automatically.

```
int x = 10;
double y = x;
```

Here the integer value can be converted to a double automatically.

Explicit type conversion / type casting: the programmer tells C which type to use.

```
double num = 10.75;
int rounded = (int) num;
```

The cast `(int)` converts the value to an integer representation, so the fractional part is discarded and the result becomes 10.

> **Memory trick:** Implicit = computer converts automatically. Explicit = programmer tells the computer to convert.

PAGE 180 — Programme Example 4.2 and Questions

The textbook combines variables, data types, a constant using `#define`, a constant using `const`, and type casting in one programme.

```
#include <stdio.h>
#define PI 3.1416

int main()
{
int age = 16;
float height = 5.9;
char grade = 'A';
const int MAX_SCORE = 100;
double num = 10.75;
int rounded = (int) num;

printf("Age: %d\n", age);
printf("Height: %.1f meters\n", height);
printf("Grade: %c\n", grade);
printf("Max Score: %d\n", MAX_SCORE);
printf("Rounded Number: %d\n", rounded);

return 0;
}
```

How to teach the programme:

- `#define PI 3.1416` creates a named fixed value.
- `int age = 16;` stores a whole number.
- `float height = 5.9;` stores a decimal.
- `char grade = 'A';` stores one character.
- `const int MAX_SCORE = 100;` creates a value intended not to change.
- `int rounded = (int) num;` demonstrates explicit type casting.

The page then asks students to choose suitable data types for age, weight and an initial; write a program using int/float/char; explain why const or #define is used; find the output of a small program; and write a program using PI to calculate the area of a circle.

Example answer for the circle task:

```
#include <stdio.h>
#define PI 3.14

int main()
{
float radius = 5;
float area = PI * radius * radius;

printf("Area = %.2f\n", area);
return 0;
}
```

For radius 5, area = 3.14 × 5 × 5 = 78.5 square units.

PAGE 181 — Operators and Expressions

An **operator** is a special symbol used to instruct the computer to perform a mathematical or logical operation.

The textbook groups C operators into arithmetic, relational, logical, bitwise, assignment, and miscellaneous operators.

Arithmetic Operators

Operator	Meaning	Example idea
+	Addition	20 + 30 = 50
-	Subtraction	20 - 30 = -10
*	Multiplication	20 * 30 = 600
/	Division	30 / 20 = 1.5 when treated as decimal values
%	Modulus/remainder	30 % 20 = 10

Important: The modulus operator gives the remainder after division.

```
10 % 3 // remainder is 1
20 % 5 // remainder is 0
```

> **Simple example:** If 10 sweets are shared among 3 students, each gets 3 and 1 sweet remains. The remainder is 1.

Arithmetic Expression Example

Suppose A = 20 and B = 30. Then addition, subtraction, multiplication, division and modulus can be used to calculate different results.

Also note that integer division behaves differently from decimal division. For example, `5 / 2` with two int operands gives 2 in ordinary C integer arithmetic, because the fractional part is not stored in an int result.

PAGE 182 — Increment, Decrement and Arithmetic Programme Example 4.3

The page introduces ++ and -- as operators that increase or decrease an integer value by one.

Increment ++: increases a value by 1.

```
int x = 10;
x++; // x becomes 11
```

Decrement --: decreases a value by 1.

```
int x = 10;
x--; // x becomes 9
```

Programme Example 4.3

The textbook programme uses `x = 55` and `y = 48`, then performs addition, subtraction, multiplication, division, modulus, increment and decrement operations.

```
#include <stdio.h>

int main()
{
int x = 55;
int y = 48;
int z;

z = x + y;
printf("Line 1 - Value of z is %d\n", z);

z = x - y;
printf("Line 2 - Value of z is %d\n", z);

z = x * y;
printf("Line 3 - Value of z is %d\n", z);

z = x / y;
printf("Line 4 - Value of z is %d\n", z);

z = x % y;
printf("Line 5 - Value of z is %d\n", z);

z = x++;
printf("Line 6 - Value of z is %d\n", z);

z = x--;
printf("Line 7 - Value of z is %d\n", z);

return 0;
}
```

Teaching point: Because x and y are integers, `55 / 48` gives 1 in integer division, while `55 % 48` gives the remainder 7.

PAGE 183 — Relational Operators

Relational operators compare two values. The result of the comparison is treated as true or false.

Operator	Meaning	Simple question
==	Equal to	Are the two values equal?
!=	Not equal to	Are the two values different?
>	Greater than	Is the left value greater?
<	Less than	Is the left value smaller?
>=	Greater than or equal to	Is the left value at least as large?
<=	Less than or equal to	Is the left value no larger?

Very important: Do not confuse = with ==. = assigns a value; == compares two values.

```
int age = 20; // assignment
age == 20 // comparison
```

Suppose A = 20 and B = 30: A == B is false; A != B is true; A > B is false; A < B is true; A >= B is false; A <= B is true.

PAGE 184 — Relational Operators Programme

The textbook demonstrates relational operators with `a = 21` and `b = 10`, using `if` and `else`.

```
#include <stdio.h>

int main()
{
int a = 21;
int b = 10;

if (a == b)
printf("a is equal to b\n");
else
printf("a is not equal to b\n");

if (a < b)
printf("a is less than b\n");
else
printf("a is not less than b\n");

if (a > b)
printf("a is greater than b\n");
else
printf("a is not greater than b\n");

a = 5;
b = 20;

if (a <= b)
printf("a is either less than or equal to b\n");

if (b >= a)
printf("b is either greater than or equal to a\n");

return 0;
}
```

Explain if/else: “if” checks a condition. If it is true, the first block runs. “else” runs when the condition is false.

For a = 21 and b = 10: a == b is false; a < b is false; a > b is true. After changing the values to a = 5 and b = 20, a <= b is true and b >= a is true.

PAGE 185 — Logical Operators

The textbook lists the three logical operators: && (AND), || (OR), and ! (NOT). It uses A = 1 and B = 0 to illustrate them.

A. Logical AND (&&)

Meaning: The condition is true when both operands are non-zero.

Simple rule: BOTH must be true.

A	B	A && B
0	0	0 (FALSE)
0	1	0 (FALSE)
1	0	0 (FALSE)
1	1	1 (TRUE)

B. Logical OR (||)

Meaning: The condition is true when either of the two operands is non-zero.

Simple rule: AT LEAST ONE must be true.

A	B	A || B
0	0	0 (FALSE)
0	1	1 (TRUE)
1	0	1 (TRUE)
1	1	1 (TRUE)

C. Logical NOT (!)

Meaning: NOT reverses the logical state of its operand. A true condition becomes false, and a false condition becomes true.

```
!(A && B)
```

For the textbook example A = 1 and B = 0: A && B is false, A || B is true, and NOT can reverse a condition.

PAGE 186 — Logical Operators Programme and MinGW

The textbook programme uses values such as `a = 5` and `b = 20` to demonstrate logical conditions and then shows how to compile and run the program using MinGW.

```
#include <stdio.h>

int main()
{
int a = 5;
int b = 20;

if (a && b)
printf("Condition is true\n");

if (a || b)
printf("Condition is true\n");

if (a && b)
printf("Condition is true\n");
else
printf("Condition is not true\n");

if (!(a && b))
printf("Condition is true\n");

return 0;
}
```

Key point: In C, 0 is false and a non-zero integer is treated as true in a condition. Therefore both 5 and 20 are logically true.

Compiling and Running with MinGW

The page explains that the program can be compiled from Command Prompt. If the source file is called `logical2.c`, a typical command is:

```
gcc logical2.c -o logical2
```

Then run the executable from the correct directory. If the file is on the Desktop, use a command such as `cd Desktop` first, then compile.

Simple process:

C source file → GCC/MinGW compiler → executable file → run the executable.

Assignment Operators — Introduction

The page then begins the assignment-operator section. Assignment operators are used to give values to variables.

PAGE 187 — Assignment Operators

The basic assignment operator is =. Compound assignment operators combine an arithmetic/bitwise operation with assignment.

Operator	Example	Equivalent idea	Meaning
=	a = b	a = b	Assign b to a
+=	a += b	a = a + b	Add b to a
-=	a -= b	a = a - b	Subtract b from a
*=	a *= b	a = a * b	Multiply a by b
/=	a /= b	a = a / b	Divide a by b
%=	a %= b	a = a % b	Store the remainder

Easy explanation: Compound assignment operators are shortcuts.

```
a = a + b;
a += b;
```

These two statements perform the same basic operation: add b to a and store the result in a.

Example

```
int a = 28;
int b = 5;

a += b; // a becomes 33
a -= b; // a becomes 28
a *= b; // a becomes 140
a /= b; // a becomes 28
a %= b; // a becomes 3
```

The exact value at each step depends on the value left in a by the previous operation. This is a useful example for showing students that an assignment changes the current value of a variable.

PAGE 188 — Operator Precedence

Operator precedence defines how operators in an expression are grouped and evaluated. Operators with higher precedence are evaluated before operators with lower precedence.

The textbook's key example is:

```
x = 8 + 4 * 3;
```

Multiplication has higher precedence than addition. Therefore C first calculates 4 × 3 = 12, then 8 + 12 = 20. So x becomes 20, not 36.

Simple Precedence Order to Teach First

Priority	Operators / group	Easy meaning
High	( )	Parentheses first
High	++ -- and other unary operators	Unary operations
High	* / %	Multiplication, division, remainder
Next	+ -	Addition and subtraction
Next	< > <= >= == !=	Comparisons
Next	&&	Logical AND
Next	||	Logical OR
Low	= and other assignment operators	Assignment

The full textbook precedence table also includes postfix, unary, shift, bitwise, conditional and comma operators, with associativity rules. For beginners, the main lesson is: higher precedence is evaluated first, and parentheses can make the intended order clear.

Associativity

Associativity tells C which direction to use when operators of the same precedence occur together. Many arithmetic operators are evaluated from left to right, while some unary and assignment operators associate from right to left.

PAGE 189 — Operator Precedence Programme Example 4.7

The textbook uses values a = 40, b = 20, c = 10 and d = 5 to demonstrate how parentheses and precedence affect the result.

```
int a = 40;
int b = 20;
int c = 10;
int d = 5;
int e;

e = (a + b) * c / d;
e = ((a + b) * c) / d;
e = (a + b) * (c / d);
e = a + (b * c) / d;
```

Example 1: (40 + 20) × 10 ÷ 5 → 60 × 10 ÷ 5 → 600 ÷ 5 → **120**.

Example 2: ((40 + 20) × 10) ÷ 5 → 600 ÷ 5 → **120**.

Example 3: (40 + 20) × (10 ÷ 5) → 60 × 2 → **120**.

Example 4: 40 + (20 × 10) ÷ 5 → 40 + 200 ÷ 5 → 40 + 40 → **80**.

> **Teacher tip:** Tell students to solve a complicated expression one small step at a time. Parentheses are useful because they make the intended order obvious.

PAGE 190 — Increment and Decrement Operators

The increment operator ++ increases a variable by 1. The decrement operator `--` decreases a variable by 1.

A. Prefix Increment (++a)

Rule: Change first, then use the value.

```
int a = 15;
printf("%d\n", ++a);
```

First a becomes 16. Then 16 is printed. Output: **16**.

B. Postfix Increment (a++)

Rule: Use first, then change the value.

```
int a = 15;
printf("%d\n", a++);
```

First 15 is used/printed. Then a becomes 16. Output: **15**.

C. Prefix Decrement (--a)

Rule: Change first, then use the value.

```
int a = 15;
printf("%d\n", --a);
```

First a becomes 14. Then 14 is printed. Output: **14**.

D. Postfix Decrement (a--)

Rule: Use first, then change the value.

```
int a = 15;
printf("%d\n", a--);
```

First 15 is used/printed. Then a becomes 14. Output: **15**.

Best memory trick: ++a = change first, use later. a++ = use first, change later. The same rule applies to `--a` and `a--`.

PAGE 191 — Exercise 4.6 and Introduction to Control Flow

The page contains revision questions on operator precedence, arithmetic operators, increment/decrement, logical conditions, modulus, and postfix decrement. It then introduces the next topic: Control Flow in C.

Exercise 4.6 — Suggested Answers

1. What is the output?

```
int a = 5, b = 3;
int result = a + b * 2;
printf("%d\n", result);
```

Multiplication comes before addition: 3 × 2 = 6, then 5 + 6 = 11. **Answer: 11.**

2. Program to input two numbers and print sum, difference, product and quotient:

```
#include <stdio.h>

int main()
{
int a, b;

printf("Enter two numbers: ");
scanf("%d %d", &a, &b);

printf("Sum = %d\n", a + b);
printf("Difference = %d\n", a - b);
printf("Product = %d\n", a * b);
printf("Quotient = %d\n", a / b);

return 0;
}
```

3. What is the output?

```
int num = 5;
printf("%d\n", num++);
printf("%d\n", ++num);
```

First `num++` prints 5 and then num becomes 6. Next `++num` increases it to 7 before printing. **Output: 5 then 7.**

4. Check whether a number is greater than 10 and less than 20:

```
#include <stdio.h>

int main()
{
int number;
scanf("%d", &number);

if (number > 10 && number < 20)
printf("In range");
else
printf("Out of range");

return 0;
}
```

The && means both conditions must be true.

5. Modulus/division question

The scanned question is not completely clear in the source image, so the exact original code should be checked against the textbook before marking students' answers. The safe teaching point is to apply operator precedence and remember how integer division and modulus work.

6. If a = 100 and b = a--, what are the final values?

```
a = 100;
b = a--;
```

Because `a--` is postfix, b receives the old value first: b = 100. Then a decreases to 99. **Final values: a = 99, b = 100.**

Introduction to Control Flow in C

The page begins the next topic by explaining that **control flow** determines how a C program executes instructions. It helps programmers control program execution using conditions and make decisions based on user input or other conditions.

Very simple explanation: “Control flow tells the computer what instruction to do, when to do it, and sometimes how many times to repeat it.”

Examples that students will meet in the following control-flow lessons include decisions using conditions and repetition using loops. **Note:** The detailed sections on loops, including break and continue, are beyond the requested page range 174–191 and are therefore not included in this PDF.

ONE-PAGE REVISION SHEET — PAGES 174–191

Topic	Remember this
Variable	A named memory location used to store a value.
Declaration	Create a variable and specify its type: int age;
Initialisation	Give the first value: int age = 16;
Updating	Change an existing value: age = 20;
const	A value intended not to be changed after initialisation.
int	Whole numbers.
float	Decimal numbers with single precision.
double	Decimal numbers with higher precision.
char	One character.
Implicit conversion	Compiler converts automatically.
Explicit casting	Programmer requests conversion, e.g. (int)num.
Arithmetic	+ - * / %
Relational	== != > < >= <=
Logical	&& || !
Assignment	= += -= *= /= %=
Precedence	Higher-priority operators are evaluated first.
Prefix ++/--	Change first, then use.
Postfix ++/--	Use first, then change.
Control flow	Controls the order in which program instructions execute.

KEY CLASSROOM MEMORY RULES

```
&& → BOTH must be true
|| → AT LEAST ONE must be true
! → REVERSE true/false
= → ASSIGN
== → COMPARE
++a → CHANGE FIRST, USE
a++ → USE FIRST, CHANGE
* / % → before + -
```

These notes are based on the uploaded scanned textbook pages 174–191. The source is image-based, so some original code on the scan is difficult to read perfectly; where that occurred, the notes use a clean equivalent C example rather than claiming an unclear scan says something exact.